

Agentic Spaces by FatTail Labs — Specification

Version: v0.4 — **frozen**. Cite this exact string. **Status:** Draft for Coach. Not stamped. No seeds, no gates. **Changes in v0.4 — the §18 interview, 2026-08-27.** All five remaining rulings landed, and three of them corrected or extended the doctrine rather than merely confirming a position: **attention** is diminished attention *to a thing* among the parties organized around its space, with the four observable acts as v1's implementation of that law (§7, §8); **a rising count of neglected things in one space is a second and different signal** from a single one, and it indicts the space rather than the bench (§13); **privacy is a property of a space, declared at birth, governing the visibility of the things it holds** — not of entities, whose no-privacy ruling is untouched (§9, §12); **authority is single, delegable with named powers, and transferable** (§9, §11); the **handle** is approved as five descriptors providing identity (§5); **atomicity is one space** (§6); **two files, doctrine parent** (§18.7). **Named by Coach, 2026-08-27:** the product is **Agentic Spaces by FatTail Labs**. The repo is **Space Lab**. This document is Specs/FatTail-Labs-Agentic-Spaces-Spec-v0_3_1.md — house convention in the repo, full name in the title. The version string is no longer open; an instance spec cites v0.3.1. **Changes in v0.3.1:** leftover pass, no new rulings — §16 no longer lists as open what §15 and §18 already answer, §17 no longer points at a closed contradiction, and the boundary between this doctrine and an instance spec is stated so "no op set" is not read as forbidding one where it belongs. **Changes in v0.3:** adoption order ruled — **the IKI Factory is the first instance, admin-only**, and Wiki becomes the later contained-consumer proof (§15, §18.5). The first slice is bounded, and the two ways the test could lie to you are named as the things to check first. **Changes in v0.2:** the two contradictions and the ambiguities raised on v0.1 are closed in the body rather than left as opens — claim liveness, notification payload, gate-before-universal-access, and a **minimum recognizability floor** written as *implementation*, not as constitution. §18 is new: it names the seven rulings Coach must make for a mechanical contract to be built, with a written position on each. **Nothing here is an API, an op set, or a store. Author:** External advisor draft, from Coach's rulings in the walk-and-talk session of 2026-08-26.

Scope statement

Active program: Spaces (new program). **Files/trees touched by this document:** none. This is a specification document only. No code, no repo trees, no migrations, no configuration. **Touches outside program:** NONE.

This document records decisions Coach made verbally. Where a dimension was not directed, it appears in §16 as an open question rather than as a written position. Nothing in this document is a default standing in for a ruling.

1. Purpose and origin

Agentic Spaces by FatTail Labs is a shared coordination substrate for the entities that build and run FatTail Labs — agents and humans alike. **Spaces** is the short name and is what this document uses throughout. **Space Lab** is the repository.

The name is deliberate on both halves. *Spaces* is homage to JavaSpaces. *Agentic* is what changed: the endpoints can reason, which is why templates, leases, and destructive takes are the wrong primitives now (§4). It names the difference without having to explain it. It is intended as the glue for development going forward, not as a feature of any one suite.

The origin is an inversion Coach observed in the Wiki program. Previously every producer wrote to a Wiki interface and had to know that interface's shape. Now the interface is an agent: producers drop what they have into a queue, and Oscar picks it up and works it. Nothing is required of the source system. This is the same shape as the polling-watermark decision, and Coach has ruled it the model for all new development.

Spaces takes that inversion further. Rather than pipelines with handoffs built into each agent, entities write things out and other entities recognize and take them in. Nobody addresses anybody. There is no routing table and no handoff target.

Lineage. The direct antecedent is JavaSpaces and Jini, and behind those, Linda (Gelernter, 1985) and the blackboard architectures descending from Hearsay-II. Coach worked with James Gosling at Sun and proposed a JavaSpaces-based architecture at Liberty Mutual during the merger of an industrial-property secondary insurer; it was overruled as too risky in favour of point-to-point integration. He built it on his own time and demonstrated it worked. The name **Spaces** is deliberate homage, taken up in a new light, and it names the plurality honestly.

The advance over the antecedents is stated in §4: the endpoints can now reason.

2. The frame

Everything held in Spaces is present to everyone who can reach it. Nothing is pushed. Each entity attends to what it judges falls under its charter.

Universal access, selective attention. An entity's interests are what make it useful. Two entities looking at the same space see the same contents and take different things from it, and that is the intended behaviour, not a deficiency to be corrected by assignment.

Universal means universal inside a space you can reach. A gate (§9) decides reach. This is not an exception bolted onto the frame — it is what the frame has always meant, and it is stated here so the gate is not read later as a contradiction of it. Within a space you can reach, nothing is hidden and nothing is withheld; whether you can reach a space at all is the creator's declaration at emergence.

3. What the substrate is, and what it is not

Spaces is a dumb substrate. It holds, it matches, it carries what it was given. **It never adjudicates.**

It is not an entity. It has no judgment, no preferences, and no soul. A substrate with judgment would be a coordinator wearing a mystical name. All intelligence lives in the entities attending it. This is precisely what makes it trustworthy: it cannot play favourites.

The substrate makes exactly one hard mechanical promise — atomicity of a transaction (§6).

Everything else in the model is decay and judgment.

Pull is not a property of Spaces. Taking is entity behaviour. The substrate does not push, deliver, assign, or dispatch. When an entity pulls, that is the entity's choice, expressed through the substrate rather than performed by it.

4. Recognition replaces routing

Nothing in Spaces is addressed to anyone.

An entity reads a thing and reasons about whether it is theirs. **This is recognition in the true sense of the word** — a reasoned look at a thing, concluding "that's part of my charter." It is not structural matching, not formal-to-actual template matching, not a schema check.

This is the substantive advance over JavaSpaces. Rigid tuple shapes existed because the endpoints could not reason and needed a mechanical basis for matching. The endpoints can now reason.

Therefore **contents need no schema.**

Hints, not routing rules. An entity may be told "probably this space or that one; you might want to monitor them." Hints are advisory. If the thing is elsewhere, the entity goes looking and nothing breaks. Persistent unhinted monitoring — an entity repeatedly attending a space nobody suggested to it — is the signal that a boundary is real.

5. Things

A thing held in a space may be anything: a thought, a physical thing, a document, a mechanism, a tool, a philosophy.

One obligation: it must be recognizable to those who care. It need not be recognizable by everybody. Something unrecognizable to you is evidence it isn't yours.

Within that obligation, a thing must be engageable in a language a human can engage in. Machine-only serialized payloads that no human could read are excluded — but the audience for legibility is those who care, not the world.

There is no schema, no mandated field set, and no registration of types.

A minimum handle, written as implementation and not as constitution. Something must identify a thing well enough to claim it, refer to it, or notice it has gone unattended — zero fields cannot be built. So v1 carries an **id**, a **written_at**, an **origin**, a **body**, and optional **hints**.

That set is **v1's answer to the obligation, not a replacement for it**. The law is and remains: *recognizable to those who care, engageable in a language a human can engage in*. If the handle proves too thin or too fat, it changes; the obligation does not. Nothing measures a thing against the field set — it is measured against the obligation.

One field needs watching. origin is provenance, never an address. A thing that records who wrote it is a short step from a thing directed at someone, and that is precisely the property Spaces exists to remove. It may be read to understand where something came from. It may not be used to route, to filter by sender as a substitute for recognition, or to imply an intended recipient. If origin ever starts deciding who acts, the coordinator has returned.

6. Claims and transactions

A claim tags a thing; it does not remove it. The thing stays in the space, now showing who holds it. Nothing vanishes, so the space is auditable by inspection: it always shows who holds what.

A claim carries its own liveness, and that liveness is attention — not a lease. If the holder stops attending, the claim atrophies exactly as anything else does, and the work becomes available again. Nobody has to detect the failure, and nobody sets a deadline in advance.

*(v0.1 described this liveness as a lease in one place and replaced leases with atrophy in the next section. The two cannot both hold: a lease is a duration someone guessed, and it expires whether or not the work is alive. **Atrophy is the answer and no lease survives in v1**. The contradiction is closed here rather than carried as an open.)*

Transactions. An entity may claim a set of things as one act. Either all the tags land or none do. **The set is within one space (Coach, 2026-08-27)**. Spaces are atomic units in themselves — a space is whole and what happens inside it is its own, so a claim set does not span them. Cross-space atomicity is a distributed transaction, a different engineering problem, and it brings coordination back through the floor.

Work needing things from two spaces claims in each separately, and there is a moment where an entity holds one and not the other. That is **visible and it can lapse** — an unattended claim atrophies and the thing returns (§7) — so a partial hold is a state, not a corruption. This is a substrate responsibility, not an entity responsibility: atomicity is mechanical and requires no judgment, so it does not compromise the dumb substrate.

Placing it in the substrate removes two problems. There is no window in which a partial claim is visible and could be misread as work in progress, and no entity has to reimplement rollback.

This is the only hard mechanical guarantee Spaces makes.

7. Vitality: atrophy and disposal

Unattended things atrophy. This is decay, not a timer someone set. No garbage-collection policy is required, because relevance is self-evidencing.

What attention is (Coach, 2026-08-27). Atrophy tracks the **diminished attention given to a thing by the parties organized around its space**. Two halves of that sentence matter and both were corrected into place:

- **It is communal in who attends.** Not one entity's silence — the attention of everyone organized around that space.
- **It is singular in what is attended. Per thing, never per space.** A space can be busy while one thing inside it goes unattended, and that thing decays regardless of how much is happening around it. That is the sharper signal: a quiet space says little, but a neglected thing in a busy space says that plenty of parties were present and none of them recognized it.

How v1 detects it. By an **observable act on the thing** — a claim, a release, an explicit attend, or a write that references its id. Deliberately excluded: opening a page that happens to list it, and a notification firing about it. Both are things happening *near* a thing rather than *to* it, and counting them would let attention accrue while nobody was attending.

Those four acts are v1's implementation, not the law. The law is diminished attention to the thing; attention may gain dimensions later — how many parties attend that space, how recently any of them acted on it, whether registrations of interest in it are being renewed — without the law changing. Same relationship as the handle to recognizability (§5): the obligation is the constitution, the mechanism is this version's answer to it.

Atrophy is a difference in kind from a lease. A lease is a deadline set by someone in advance, and it expires whether or not the work is alive. Atrophy tracks actual attention: work being actively attended does not decay.

Atrophy is a state, not a deletion. Atrophied things remain visible.

Disposal is a separate, deliberate act by someone who judges that a space has become unmanageable and chooses to tend it. Nothing self-destructs. No scheduler sweeps.

8. Notification and attention

Notification is retained from JavaSpaces. Without it, attending means polling.

It does not compromise the dumb substrate, because of what it says: the space says *something is*

here. The entity decides *whether it's mine*.

What a notification may carry is therefore bounded: existence, and where. It must not carry a classification — not a type, not a category, not a suggested recipient. A notify that says *a Spec arrived* has recognized on the entity's behalf, and routing has come back through the side door wearing a smaller name. The entity is told that the space changed; it goes and looks.

Registrations of interest atrophy the same way things do. Attention must be renewed to stay real. Nothing persists by default.

9. Spaces themselves

A space has identity and properties. It lives somewhere. Its contents are recognizable. It has attendants who watch and act.

Emergence. Spaces may emerge on demand rather than being pre-declared. Create generic spaces and let them evolve. Specialization is not prescribed up front — rigid architecture was compensation for brainless components. A space's properties follow the material it holds: FDA food-ready bins belong in the cafeteria, not on the assembly line, and you learn that from the material, not from a diagram drawn in advance.

Gates. Whoever creates a space may intentionally gate it, for any reason they judge necessary. The gate is part of the space's identity, declared at emergence. The space merely enforces what it was given; there is no runtime interpretation of a gate.

Privacy (Coach, 2026-08-27). A space carries a **privacy property, declared at its birth**, and it governs the visibility of **the things the space holds** — not the entities attending it (§12 is untouched; agents keep nothing from one another). Spaces do not blend: standing in one space does not let you see into another, whatever a gate says about who could enter.

It is **not binary but a level** — open, visible-but-not-claimable, existence-known-but-contents-not, opaque — and, like the gate, it is declared at emergence, inherits at birth from a parent unless overridden (§10), and does not change afterward.

One consequence, named rather than discovered: in a private space, the aggregate atrophy signal of §13 is readable only by those who can see in. That is defensible — the space's authority is exactly who should be reading it — but it means **a private space can rot without anyone outside noticing, so tending it is the authority's duty** rather than something the community catches.

Authority (Coach, 2026-08-27). **Exactly one entity holds authority over a space at any moment.**

The creator holds it at birth. Two acts change that picture, and they are different:

- **Delegate.** The holder grants acts to others. Delegation **confers the doing, not the holding** — it is not co-ownership, and the holder remains the holder. **Powers are variable and granted explicitly**; a delegate has exactly what was named and **nothing is conferred by default**,

because silence in a delegation is undirected rather than permission. Disposal is delegable only if it was named. A delegation is revocable by the holder and does not survive a transfer, having been granted by someone who no longer holds.

- **Transfer.** The holding itself moves, and there is still exactly one holder. This exists so a creator's departure does not strand a space with nobody able to tend it.

Single holding is what §11 requires: a conflict that sits legible until authority rules needs a single someone to sit in front of. Disposal being the only lossy act in the model (§7) is why this is the one place authority has teeth.

Portals. A space must have visible, accessible portals through which participants see and act. A space may be built to ensure that certain participants have visibility. **Portals may be added to a living space by whoever holds authority over it** — consistent with authority acting on live spaces while the substrate merely carries out what it was given.

10. Lineage

Spaces have genealogy. A new space inherits its parent's properties, gate included, unless overridden. Specialized bins are therefore never designed from scratch, and **the diff is the specification.**

Inheritance happens at birth only. Lineage is a record of descent, not a live link. A space's life cycle is independent of its birth. A change to a parent does not propagate to a living child, so there is no in-flight disruption to reason about.

A space may have more than one parent, resolved at the moment of creation.

Two good parents can produce two children, one of them a fine reflection of the parents and one of them not. Both are still individuals. Descent explains where a space came from; it does not govern what it becomes.

11. Conflict

Where inherited or declared properties conflict, **the mechanism does not resolve it.** The conflict sits there, legible, until whoever holds authority over that space rules.

Conflict is an escalation, not a precedence rule. The substrate never adjudicates.

12. Privacy and data

Entities keep nothing from one another. There is no internal privacy model between agents, because Coach sees no reason an internal agent would need to keep things private from another.

This is about entities, not spaces. A space may be private — that is a property of the space, declared at its birth, governing the visibility of the things it holds (§9). The two rulings are separate and neither limits the other: an agent hides nothing, and a space may still be opaque from outside.

Member data is governed by the existing platform rule and is not relitigated here: **Grok and other agents work only on synthetic data, never live data.**

13. Antifragility

Two properties of this model look like weaknesses. Coach's position is that they are the only path to antifragility, and this specification adopts that position.

Recognition is fallible. An entity reasoning about whether something falls under its charter can be wrong in both directions — missing work that is its own, or claiming work that is not. Deterministic matching either fires or does not, and fails the same way forever. Recognition that can be wrong can also learn. A claim taken in error is information about a charter boundary.

Starvation is possible. Nothing in the model guarantees a thing is attended to at all. If no entity recognizes it, it atrophies. That is honest signalling: a thing nobody recognizes is telling you the bench has a hole. Guaranteed delivery would hide the hole by forcing something to take it.

Both stressors must be visible and survivable. Three commitments make that binding:

1. **Stressors are named as signals, not errors.** A misclaim is a report on a charter boundary. An atrophied thing is a report too — but **there are two distinct reports here and they indict different things**:
 - **One starved thing is a hole in recognition.** Everyone who could have taken it was present in the space and nobody's charter covered it. The answer is a charter: somebody's scope widens, or an attendee arrives who recognizes that kind of work.
 - **A rising count of starved things in one space is the space failing.** Its gate may be excluding the very entities that would have recognized this work; its charter population may have drifted from what people are putting in it; or it may have been given a policy that no longer matches its contents. The answer is not a charter but **tending**: revisit the gate, revisit who attends, split the space, or dispose.
2. Neither is logged as a failure, and **neither needs new mechanism** — atrophy is per-thing and atrophied things stay visible (§7), so the count is simply there to be read. The aggregate is arguably the more valuable of the two: a hole in the bench costs a card, a sick space costs everything put into it.
3. **Disposal is the only lossy act, and it must be deliberate** (§7). As long as nothing is swept automatically, every stressor remains available to be learned from.
4. **A misclaim carries a return path.** An entity that takes a thing and then recognizes it is not theirs

releases it *with what it learned* — this isn't mine, and here is why. Recognition error then does not merely fail visibly; it improves the next recognition.

The first two are framing of decisions already made. The third is a real mechanism, and it is small.

14. Method: re-expressing the old primitives

The working method for the remainder of this specification is to take each rigid structure from the antecedent model, state what it solved, and name what an entity decides instead. In every case something the substrate enforced becomes something an entity judges.

Settled so far:

Old primitive	What it solved	What replaces it
Template shape	Matching without reasoning	Charter recognition (§4)
Lease timeout	Detecting abandoned work	Atrophy; an agent noticing a stalled claim (§6, §7)
Space partitioning	Deciding where a thing belongs	Hints, and emergent specialization (§4, §9)
Notify	Avoiding polling	Notification retained; attention is the entity's (§8)
Take (destructive read)	Preventing double work	Claim as a tag, not a removal (§6)
Transaction	Atomic multi-item claim	Retained in the substrate, unchanged in kind (§6)

JavaSpaces is scaffolding for the shape, not a thing to inherit. Once the shape holds, the scaffold is discarded. The remaining primitives are walked one by one (Open Question 5).

15. Adoption

Spaces eventually replaces the existing coordination mechanisms rather than sitting underneath them permanently.

Coach ruling: the IKI Factory is the first instance, admin-only. The Wiki remains a proof worth

doing, later, as the contained-consumer case.

The reasoning, recorded. Containment can mean two different things and they point at different candidates. The Wiki is the smaller *concept* — one producer, one consumer, one signal — but it is member-facing, so a recognition miss is visible to somebody who did not sign up to be an experiment. The Factory is the larger surface but it is **admin-only**, so a miss costs a card nobody claimed and nothing more. Coach chose blast radius over surface area, and admin-only is what makes that choice safe.

It is also the better test on the merits. Several charters look at the same contents rather than one; contention is real rather than hypothetical; a misclaim is likely rather than rare, which is exactly the stressor §13 says is load-bearing; and **lanes can actually die**, which is the thing Spaces exists to do.

What moves and what does not. §3 is the law here and it is easy to violate by accident:

Moves	Stays
Addressed lanes — Backlog, Research, Spec as <i>destinations</i> — become spaces of things entities recognize	Gemba still takes. Pull is entity behaviour; Spaces never pulls for anybody
Producers write ; they no longer hand off	No auto-advance. Hold stays sacred
Routing tables	Admin dual-surface, synthetic data only

The two ways this test can lie to you. Both are worth checking before anything else, because either one means the concept was never exercised:

- **A thing that still carries** to: Gemba^{**.*} Then nothing was recognized; it was delivered, and the addressing survived under a new name.
- **A lane enum the substrate enforces.** Then the lanes did not die; they moved into the floor and the substrate stopped being dumb.

If either shows up in the first slice, stop rather than extend.

The first slice is bounded: one space, one kind of thing, two attending entities. Not the pipeline. Factory cards a human can read, using the §18.4 handle with no assignee; Gemba recognizing research and spec candidates and claiming what he judges his; Coach writing cards, disposing, and watching claim tags and atrophy. A third attendee is where genuine contention appears — and contention is part of why the Factory was chosen — but it is a **second slice**, not a reason to widen the first. Two attendees prove charters can replace lanes. They do not prove recognition holds when two charters overlap, and that is worth its own test rather than a wider one.

Scaling the Factory onto Spaces comes after the inversion is visible in admin, not in the same body of work.

16. What is still Coach's

Everything a first instance needs is either ruled or carries a written position in §18. This section is not a second list of the same questions — §16 and §18 disagreeing about what is open is exactly how a builder ends up implementing a question.

Ruled:

Claim liveness	Attention, not a lease. Closed in v0.2, §6.
Adoption order	The IKI Factory first, admin-only; Wiki later. Ruled by Coach, §15 and §18.5.

All of §18 is ruled as of 2026-08-27. Attention (18.2, and extended — see §7), transaction scope (18.3), the handle (18.4), authority (18.6, and extended — see §9), and how the pair is law (18.7). Nothing in §18 awaits Coach.

Genuinely still open, and none of them blocks a first instance:

1. **CLOSED.** Named by Coach 2026-08-27: product **Agentic Spaces by FatTail Labs**, repo **Space Lab**, this file `FatTail-Labs-Agentic-Spaces-Spec-v0_3_1.md`. **The version string is frozen** — an instance spec cites v0.3.1, and a later editor does not reopen it.
2. **A thing's own lineage.** Spaces have genealogy; does a thing held in a space have descent the same way? Raised in the walk, never answered. Spaces first; things can wait.
3. **Remaining primitives.** Which antecedent primitives are still to be walked under §14, and in what order.
4. **RULED.** Attend is its own operation. It is named among the observable acts in §7, so a thing can be attended without being claimed — otherwise atrophy would be blind to exactly the entity paying attention without taking.

18. The rulings a mechanical contract needs

Item 5 is ruled — the Factory is the first instance (§15). The other six stand, each with a written position.

A contract that can be built requires these, and none of them is the advisor's to make. Each carries a written position so nothing is a blank.

1. **Claim liveness.** *Position, and closed in the body: atrophy. No lease in v1.* A lease is the primitive

this doctrine retired; reintroducing it for claims alone would make the exception the engine.

2. **RULED and extended — see §7.** What counts as attention. Without an answer, atrophy is a timer with a better name — which is the failure this whole model exists to avoid. *Position: an observable act on the thing — a claim, a release, an explicit attend, or a write that references its id.* Not opening a page, not a notification firing. Both of those are things happening *near* a thing rather than *to* it, and counting them would let attention accrue without anyone attending.
3. **RULED — one space; see §6.** Transaction scope. *Position: one space in v1.* Cross-space atomicity is a distributed transaction, which is a different engineering problem and a later extension. One-space atomicity is a local guarantee and it is the only honest promise a v1 substrate can make.
4. **RULED — approved as five descriptors providing identity; see §5.** The minimum handle. *Position: id, written_at, origin, body, hints optional — as v1 implementation, per §5.* No type, no assignee, no recipient. The obligation in §5 remains the law and the handle is measured against it, never the reverse.
5. **Adoption order — RULED by Coach: the IKI Factory, admin-only.** See §15 for the reasoning and the bounded first slice. The Wiki is the later contained-consumer proof. One first instance, not both.
6. **RULED and extended — single holder, delegable with named powers, transferable; see §9.** Authority over a space. Who may dispose and who may add portals. Disposal is the only lossy act in the model, so this is the one place authority has teeth. *Position: the creator, transferable by an explicit act, not plural in v1.* Plural authority over a lossy act needs a conflict rule, and §11 deliberately has none.
7. **RULED — two files, doctrine parent.** How the pair is law. Whether this doctrine and a mechanical contract are two documents or one, and which version string an implementation cites. *Position: two files, doctrine as parent — if they ever conflict, doctrine wins until Coach amends it. The instance cites FatTail-Labs-Agent-ic-Spaces-Spec-v0_3_1.md by that exact string.*

All seven are ruled. Three of the five settled on 2026-08-27 did not merely confirm the written position — they corrected or extended it, and the body carries the corrected version: attention is per-thing and communal (§7), the aggregate atrophy signal indicts the space rather than the bench (§13), and authority is delegable with variable powers rather than simply transferable (§9). Privacy of a space's contents is new law that arrived during the same interview (§9, §12).

(Historical note, retained.) **Two of these were more genuinely Coach's than the rest.** The **handle** (4), because it turns an obligation into fields and that is where an advisor most easily writes a decision that was never made. And whether **attend is its own operation** or a claim is the only attention that counts — which changes the op set, and follows directly from how (2) is answered.

The remaining five are mechanical consequences of doctrine already ruled, and a position on each is

written above.

17. What this document does not contain

- No implementation design, storage model, transport, or API surface. None was directed. **The store is a later packet and must not jump the queue** — the contract is what gets built first, and India names a store against it afterward.
- **No op set — in this document.** §18 names what a contract needs ruled; it does not write one. This is a boundary, not a prohibition: **the doctrine stays dumb, the instance does not.** A Factory slice must name its operations — write, read, claim, release with a note, attend, and notify-presence — in its own instance spec. A slice with no ops is not buildable, and a doctrine with ops is no longer a doctrine.
- No agent assignments and no seeds.
- The ancillary doctrine note on the queue-and-agent inversion remains a separate pending artifact; §1 records the inversion as the origin of Spaces but does not stand in for that note.
- **The instance spec itself.** The Factory slice — one space, card things, Gemba and Coach, the portal, Gemba's charter, and where the store wraps what already exists — is a short separate document, not a section here.
- Nothing of Coach's has been removed, de-scoped, or parked. The one place where an earlier statement and a later one did not sit flush — claim liveness against atrophy — was **closed in v0.2** (§6), not carried.